

OSGi Working Group  
OSGi Specification repo for  
`org.osgi.service.upnpRelease`

1.0.0-SNAPSHOT

September 2026



OSGi®

## LICENSE

### Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>".

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

#### Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

## Preface

### Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

### Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

---

---

# Table of Contents

|            |                                                          |          |
|------------|----------------------------------------------------------|----------|
| <b>1</b>   | <b>Introduction</b>                                      | <b>5</b> |
| 1.1        | Reader Level .....                                       | 5        |
| 1.2        | Version Information .....                                | 5        |
| 1.3        | References .....                                         | 5        |
| <b>111</b> | <b>Device Service Specification for UPnP™ Technology</b> | <b>7</b> |
| 111.1      | Introduction .....                                       | 7        |
| 111.2      | UPnP Specifications .....                                | 9        |
| 111.3      | UPnP Device .....                                        | 10       |
| 111.4      | Device Category .....                                    | 11       |
| 111.5      | UPnPService .....                                        | 12       |
| 111.6      | Working With a UPnP Device .....                         | 12       |
| 111.7      | Implementing a UPnP Device .....                         | 13       |
| 111.8      | Event API .....                                          | 13       |
| 111.9      | UPnP Events and Event Admin service .....                | 14       |
| 111.10     | Localization .....                                       | 15       |
| 111.11     | Dates and Times .....                                    | 15       |
| 111.12     | UPnP Exception .....                                     | 15       |
| 111.13     | Configuration .....                                      | 16       |
| 111.14     | Networking considerations .....                          | 16       |
| 111.15     | Security .....                                           | 16       |
| 111.16     | org.osgi.service.upnp .....                              | 16       |
| 111.17     | References .....                                         | 31       |

---

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

# 1 Introduction

This document contains a specification developed by the OSGi working group.

## 1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

## 1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

## 1.3 References

- [1] *OSGi Specifications*  
<https://docs.osgi.org/specification/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

# 111 Device Service Specification for UPnP™ Technology

## Version 1.2

### 111.1 Introduction

The UPnP Device Architecture specification provides the protocols for a peer-to-peer network. It specifies how to join a network and how devices can be controlled using XML messages sent over HTTP. The OSGi specifications address how code can be download and managed in a remote system. Both standards are therefore fully complimentary. Using an OSGi Framework to work with UPnP enabled devices is therefore a very successful combination.

This specification specifies how OSGi bundles can be developed that interoperate with UPnP™ (Universal Plug and Play) devices and UPnP control points. The specification is based on the UPnP Device Architecture and does not further explain the UPnP specifications. The UPnP specifications are maintained by [1] *Open Connectivity Foundation*.

UPnP™ is a trademark of the UPnP Implementers Corporation.

#### 111.1.1 Essentials

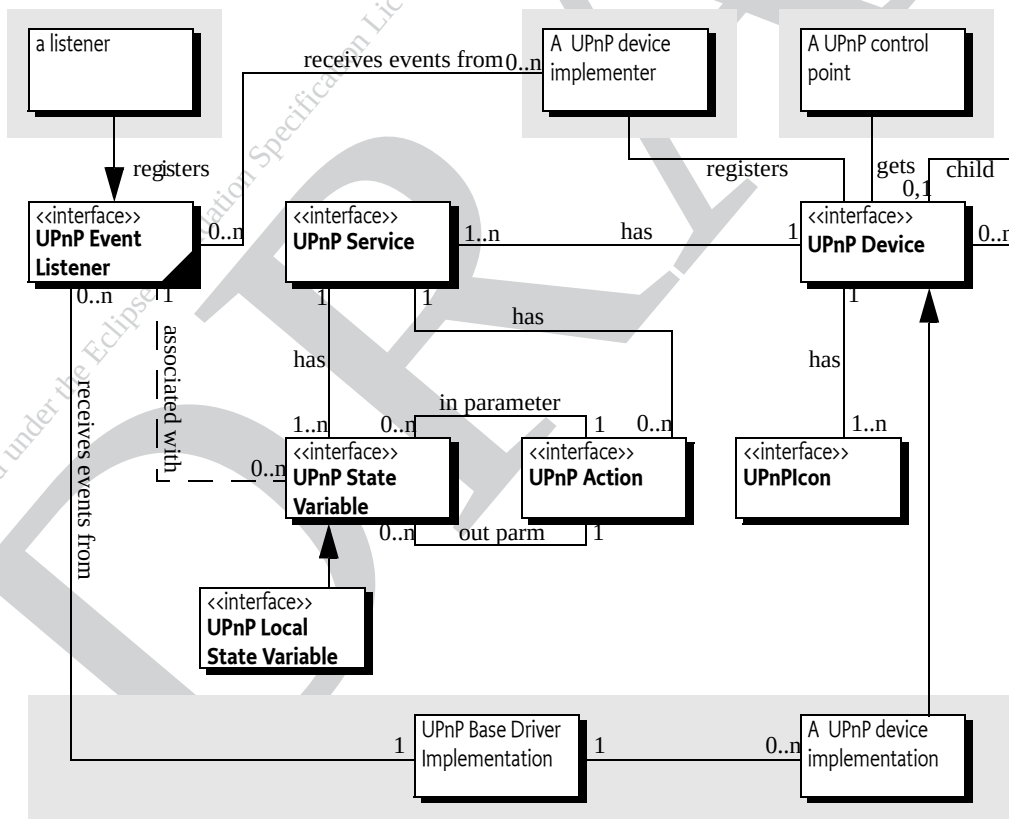
- *Scope* - This specification is limited to device control aspects of the UPnP specifications. Aspects concerning the TCP/IP layer, like DHCP and limited TTL, are not addressed.
- *Transparency* - OSGi services should be made available to networks with UPnP enabled devices in a transparent way.
- *Network Selection* - It must be possible to restrict the use of the UPnP protocols to a selection of the connected networks. For example, in certain cases OSGi services that are UPnP enabled should not be published to the Wide Area Network side of a gateway, nor should UPnP devices be detected on this WAN.
- *Event handling* - Bundles must be able to listen to UPnP events.
- *Export OSGi services as UPnP devices* - Enable bundles that make a service available to UPnP control points.
- *Implement UPnP Control Points* - Enable bundles that control UPnP devices.

#### 111.1.2 Entities

- *UPnP Base Driver* - The bundle that implements the bridge between OSGi and UPnP networks. This entity is not represented as a service.
- *UPnP Root Device* - A physical device can contain one or more root devices. Root devices contain one or more devices. A root device is modeled with a UPnPDevice object, there is no separate interface defined for root devices.
- *UPnP Device* - The representation of a UPnP device. A UPnP device may contain other UPnP devices and UPnP services. This entity is represented by a UPnPDevice object. A device can be local (implemented in the Framework) or external (implemented by another device on the net).

- *UPnP Service* - A UPnP device consists of a number of services. A UPnP service has a number of UPnP state variables that can be queried and modified with actions. This concept is represented by a UPnPService object.
- *UPnP Action* - A UPnP service is associated with a number of actions that can be performed on that service and that may modify the UPnP state variables. This entity is represented by a UPnPAction object.
- *UPnP State Variable* - A variable associated with a UPnP service, represented by a UPnPStateVariable object.
- *UPnP Local State Variable* - Extends the UPnPStateVariable interface when the state variable is implemented locally. This interface provides access to the actual value.
- *UPnP Event Listener Service* - A listener to events coming from UPnP devices.
- *UPnP Host* - The machine that hosts the code to run a UPnP device or control point.
- *UPnP Control Point* - A UPnP device that is intended to control UPnP devices over a network. For example, a UPnP remote controller.
- *UPnP Icon* - A representation class for an icon associated with a UPnP device.
- *UPnP Exception* - An exception that delivers errors that were discovered in the UPnP layer.
- *UDN* - Unique Device Name, a name that uniquely identifies the a specific device.

Figure 111.1 UPnP Service Specification class Diagram *org.osgi.service.upnp* package



### 111.1.3 Operation Summary

To make a UPnP service available to UPnP control points on a network, an OSGi service object must be registered under the UPnPDevice interface with the Framework. The UPnP driver bundle must de-

tect these UPnP Device services and must make them available to the network as UPnP devices using the UPnP protocol.

UPnP devices detected on the local network must be detected and automatically registered under the UPnPDevice interface with the Framework by the UPnP driver implementation bundle.

A bundle that wants to control UPnP devices, for example to implement a UPnP control point, should track UPnP Device services in the OSGi service registry and control them appropriately. Such bundles should not distinguish between resident or remote UPnP Device services.

## 111.2 UPnP Specifications

The UPnP DA is intended to be used in a broad range of device from the computing (PCs printers), consumer electronics (DVD, TV, radio), communication (phones) to home automation (lighting control, security) and home appliances (refrigerators, coffee makers) domains.

For example, a UPnP TV might announce its existence on a network by broadcasting a message. A UPnP control point on that network can then discover this TV by listening to those announce messages. The UPnP specifications allow the control point to retrieve information about the user interface of the TV. This information can then be used to allow the end user to control the remote TV from the control point, for example turn it on or change the channels.

The UPnP specification supports the following features:

- *Detect and control a UPnP standardized device.* In this case the control point and the remote device share a priori knowledge about how the device should be controlled. The UPnP Forum intends to define a large number of these standardized devices.
- *Use a user interface description.* A UPnP control point receives enough information about a device and its services to automatically build a user interface for it.
- *Programmatic Control.* A program can directly control a UPnP device without a user interface. This control can be based on detected information about the device or through a priori knowledge of the device type.
- *Allows the user to browse a web page supplied by the device.* This web page contains a user interface for the device that be directly manipulated by the user. However, this option is not well defined in the UPnP Device Architecture specification and is not tested for compliance.

The UPnP Device Architecture specification and the OSGi Framework provide *complementary* functionality. The UPnP Device Architecture specification is a data communication protocol that does not specify where and how programs execute. That choice is made by the implementations. In contrast, the OSGi Framework specifies a (managed) execution point and does not define what protocols or media are supported. The UPnP specification and the OSGi specifications are fully complementary and do not overlap.

From the OSGi perspective, the UPnP specification is a communication protocol that can be implemented by one or more bundles. This specification therefore defines the following:

- How an OSGi bundle can implement a service that is exported to the network via the UPnP protocols.
- How to find and control services that are available on the local network.

The UPnP specifications related to the assignment of IP addresses to new devices on the network or auto-IP self configuration should be handled at the operating system level. Such functions are outside the scope of this specification.

**111.2.1****UPnP Base Driver**

The functionality of the UPnP service is implemented in a UPnP *base driver*. This is a bundle that implements the UPnP protocols and handles the interaction with bundles that use the UPnP devices. A UPnP base driver bundle must provide the following functions:

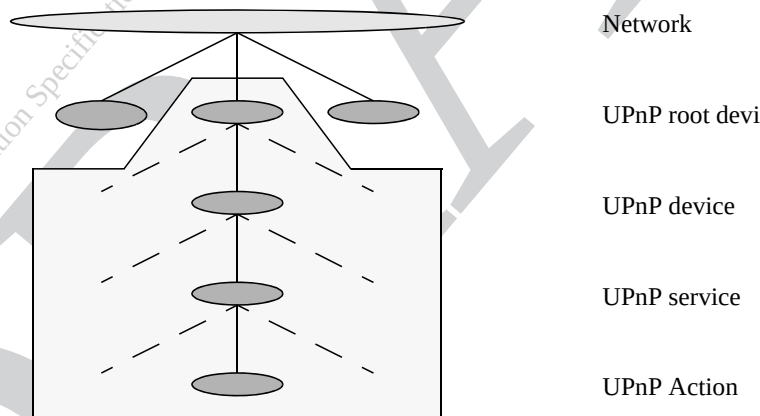
- Discover UPnP devices on the network and map each discovered device into an OSGi registered UPnP Device service.
- Present UPnP marked services that are registered with the OSGi Framework on one or more networks to be used by other computers.

**111.3****UPnP Device**

The principle entity of the UPnP specification is the UPnP device. There is a UPnP *root device* that represents a physical appliance, such as a complete TV. The root device contains a number of sub-devices. These might be the tuner, the monitor, and the sound system. Each sub-device is further composed of a number of UPnP services. A UPnP service represents some functional unit in a device. For example, in a TV tuner it can represent the TV channel selector. Figure 111.2 on page 10 illustrates this hierarchy.

Figure 111.2

UPnP device hierarchy



Each UPnP service can be manipulated with a number of UPnP actions. UPnP actions can modify the state of a UPnP state variable that is associated with a service. For example, in a TV there might be a state variable *volume*. There are then actions to set the volume, to increase the volume, and to decrease the volume.

**111.3.1****Root Device**

The UPnP root device is registered as a UPnP Device service with the Framework, as well as all its sub-devices. Most applications will work with sub-devices, and, as a result, the children of the root device are registered under the UPnPDevice interface.

UPnP device properties are defined per sub-device in the UPnP specification. These properties must be registered with the OSGi Framework service registry so they are searchable.

Bundles that want to handle the UPnP device hierarchy can use the registered service properties to find the parent of a device (which is another registered UPnPDevice).

The following service registration properties can be used to discover this hierarchy:

- **PARENT\_UDN** - (String) The Universal Device Name (UDN) of the parent device. A root device most not have this property registered. Type is a String object.
- **CHILDREN\_UDN** - (String[]) An array of UDNs of this device's children.

### 111.3.2 Exported Versus Imported Devices

Both imported (from the network to the OSGi service registry) and exported (from the service registry to the network) UPnPDevice services must have the same representation in the OSGi Framework for identical devices. For example, if an OSGi UPnP Device service is exported as a UPnP device from an OSGi Framework to the network, and it is imported into another OSGi Framework, the object representation should be equal. Application bundles should therefore be able to interact with imported and exported forms of the UPnP device in the same manner.

Imported and exported UPnP devices differ only by two marker properties that can be added to the service registration. One marker, **DEVICE\_CATEGORY**, should typically be set only on imported devices. By not setting **DEVICE\_CATEGORY** on internal UPnP devices, the Device Manager does not try to refine these devices (See the ??? for more information about the Device Manager). If the device service does not implement the Device interface and does not have the **DEVICE\_CATEGORY** property set, it is not considered a *device* according to the Device Access Specification.

The other marker, **UPNP\_EXPORT**, should only be set on internally created devices that the bundle developer wants to export. By not setting **UPNP\_EXPORT** on registered UPnP Device services, the UPnP Device service can be used by internally created devices that should not be exported to the network. This allows UPnP devices to be simulated within an OSGi Framework without announcing all of these devices to any networks.

The **UPNP\_EXPORT** service property has no defined type, any value is correct.

### 111.3.3 Icons

A UPnP device can optionally support an icon. The purpose of this icon is to identify the device on a UPnP control point. UPnP control points can be implemented in large computers like PC's or simple devices like a remote control. However, the graphic requirements for these UPnP devices differ tremendously. The device can, therefore, export a number of icons of different size and depth.

In the UPnP specifications, an icon is represented by a URL that typically refers to the device itself. In this specification, a list of icons is available from the UPnP Device service.

In order to obtain localized icons, the method **getIcons(String)** can be used to obtain different versions. If the locale specified is a null argument, then the call returns the icons of the default locale of the called device (not the default locale of the UPnP control point). When a bundle wants to access the icon of an imported UPnP device, the UPnP driver gets the data and presents it to the application through an input stream.

A bundle that needs to export a UPnP Device service with one or more icons must provide an implementation of the UPnPIcon interface. This implementation must provide an InputStream object to the actual icon data. The UPnP driver bundle must then register this icon with an HTTP server and include the URL to the icon with the UPnP device data at the appropriate place.

## 111.4 Device Category

UPnP Device services are devices in the context of the Device Manager. This means that these services need to register with a number of properties to participate in driver refinement. The value for UPnP devices is defined in the UPnPDevice constant **DEVICE\_CATEGORY**. The value is UPnP. The UPnPDevice interface contains a number of constants for matching values. Refer to **MATCH\_GENERIC** for further information.

## 111.5 UPnPService

A UPnP Device contains a number of UPnPService objects. UPnPService objects combine zero or more actions and one or more state variables.

### 111.5.1 State Variables

The UPnPStateVariable interface encapsulates the properties of a UPnP state variable. In addition to the properties defined by the UPnP specification, a state variable is also mapped to a Java data type. The Java data type is used when an event is generated for this state variable and when an action is performed containing arguments related to this state variable. There must be a strict correspondence between the UPnP data type and the Java data type so that bundles using a particular UPnP device profile can predict the precise Java data type.

The function QueryStateVariable defined in the UPnP specification has been deprecated and is therefore not implemented. It is recommended to use the UPnP event mechanism to track UPnP state variables.

Additionally, a UPnPStateVariable object can also implement the UPnPLocalStateVariable interface if the device is implemented locally. That is, the device is not imported from the network. The UPnPLocalStateVariable interface provides a [getCurrentValue\(\)](#) method that provides direct access to the actual value of the state variable.

## 111.6 Working With a UPnP Device

The UPnP driver must register all discovered UPnP devices in the local networks. These devices are registered under a UPnPDevice interface with the OSGi Framework.

Using a remote UPnP device thus involves tracking UPnP Device services in the OSGi service registry. The following code illustrates how this can be done. The sample Controller class extends the ServiceTracker class so that it can track all UPnP Device services and add them to a user interface, such as a remote controller application.

```
class Controller extends ServiceTracker {
    UI ui;

    Controller( BundleContext context ) {
        super( context, UPnPDevice.class.getName(), null );
    }
    public Object addingService( ServiceReference ref ) {
        UPnPDevice dev = (UPnPDevice)super.addingService(ref);
        ui.addDevice( dev );
        return dev;
    }
    public void removedService( ServiceReference ref,
        Object dev ) {
        ui.removeDevice( (UPnPDevice) dev );
    }
    ...
}
```

## 111.7 Implementing a UPnP Device

OSGi services can also be exported as UPnP devices to the local networks, in a way that is transparent to typical UPnP devices. This allows developers to bridge legacy devices to UPnP networks. A bundle should perform the following to export an OSGi service as a UPnP device:

- Register an UPnP Device service with the registration property `UPNP_EXPORT`.
- Use the registration property `PRESENTATION_URL` to provide the presentation page. The service implementer must register its own servlet with the Http Service to serve out this interface. This URL must point to that servlet.

There can be multiple UPnP root devices hosted by one OSGi platform. The relationship between the UPnP devices and the OSGi platform is defined by the `PARENT_UDN` and `CHILDREN_UDN` service properties. The bundle registering those device services must make sure these properties are set accordingly.

Devices that are implemented on the OSGi Framework (in contrast with devices that are imported from the network) should use the `UPnPLocalStateVariable` interface for their state variables instead of the `UPnPStateVariable` interface. This interface provides programmatic access to the actual value of the state variable as maintained by the device specific code.

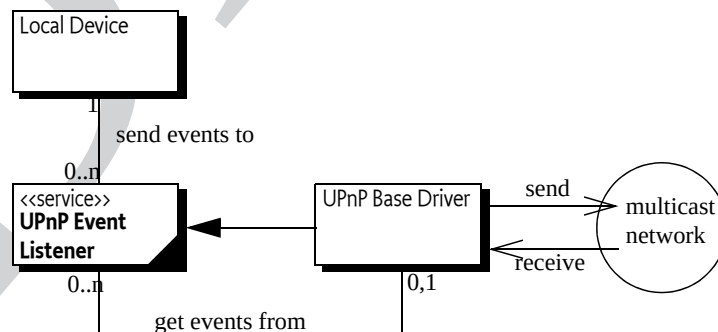
## 111.8 Event API

There are two distinct event directions for the UPnP Service specification.

- External events from the network must be dispatched to listeners inside the OSGi Frameworks. The UPnP Base driver is responsible for mapping the network events to internal listener events.
- Implementations of UPnP devices must send out events to local listeners as well as cause the transmission of the UPnP network events.

UPnP events are sent using the whiteboard model, in which a bundle interested in receiving the UPnP events registers an object implementing the `UPnPEventListener` interface. A filter can be set to limit the events for which a bundle is notified. The UPnP Base driver must register a UPnP Event Listener without filter that receives all events.

Figure 111.3 Event Dispatching for Local and External Devices



If a service is registered with a property named `upnp.filter` with the value of an instance of a `Filter` object, the listener is only notified for matching events (This is a `Filter` object and not a `String` object because it allows the `InvalidSyntaxException` to be thrown in the client and not the UPnP driver bundle).

The filter might refer to any valid combination of the following pseudo properties for event filtering:

- UPnPDevice. **UDN** - (UPnP.device.UDN/String) Only events generated by services contained in the specific device are delivered. For example: (UPnP.device.UDN=uuid:Upnp-TVEulator-1\_0-1234567890001)
- UPnPDevice. **TYPE** - (UPnP.device.type/String or String[]) Only events generated by services contained in a device of the given type are delivered. For example: (UPnP.device.type=urn:schemas-upnp-org:device:tvdevice:1)
- UPnPService. **ID** - (UPnP.service.id/String) Service identity. Only events generated by services matching the given service ID are delivered.
- UPnPService. **TYPE** - (UPnP.service.type/String or String[]) Only events generated by services of the given type are delivered.

If an event is generated by either a local device or via the base driver for an external device, the `notifyUPnPEvent(String,String,Dictionary)` method is called on all registered `UPnPEventListener` services for which the optional filter matches for that event. If no filter is specified, all events must be delivered. If the filter does not match, the UPnP Driver must not call the UPnP Event Listener service. The way events must be delivered is the same as described in *Delivering Events* of ???.

One or multiple events are passed as parameters to the `notifyUPnPEvent(String,String,Dictionary)` method. The Dictionary object holds a pair of UpnPStateVariable objects that triggered the event and an Object for the new value of the state variable.

### 111.8.1

## Initial Event Delivery

Special care must be taken with the initial subscription to events. According to the UPnP specification, when a client subscribes for notification of events for the first time, the device sends out a number of events for each state variable, indicating the current value of each state variable. This behavior simplifies the synchronization of a device and an event-driven client.

The UPnP Base Driver must mimic this event distribution on behalf of external devices. It must therefore remember the values of the state variables of external devices. A UPnP Device implementation must send out these initial events for each state variable they have a value for.

The UPnP Base Driver must have stored the last event from the device and retransmit the value over the multicast network. The UPnP Driver must register an event listener without any filter for this purpose.

The call to the listener's notification method must be done asynchronously.

### 111.9

## UPnP Events and Event Admin service

UPnP events must be delivered asynchronously to the Event Admin service by the UPnP implementation, if present. UPnP events have the following topic:

`org/osgi/service/upnp/UPnPEvent`

The properties of a UPnP event are the following:

- `upnp.deviceId` - (String) The identity as defined by UPnPDevice. **UDN** of the device sending the event.
- `upnp.serviceId` - (String) The identity of the service sending the events.
- `upnp.events` - (Dictionary) A Dictionary object containing the new values for the state variables that have changed.

## 111.10 Localization

All values of the UPnP properties are obtained from the device using the device's default locale. If an application wants to query a set of localized property values, it has to use the method [getDescription\(String\)](#). For localized versions of the icons, the method [getIcons\(String\)](#) is to be used.

## 111.11 Dates and Times

The UPnP specification uses different types for date and time concepts. An overview of these types is given in the following table.

Table 111.1 Mapping UPnP Date/Time types to Java

| UPnP Type   | Class | Example                  | Value (TZ=CEST=UTC+0200)        |
|-------------|-------|--------------------------|---------------------------------|
| date        | Date  | 1985-04-12               | Sun April 12 00:00:00 CEST 1985 |
| dateTime    | Date  | 1985-04-12T10:15:30      | Sun April 12 10:15:30 CEST 1985 |
| dateTime.tz | Date  | 1985-04-12T10:15:30+0400 | Sun April 12 08:15:30 CEST 1985 |
| time        | Long  | 23:20:50                 | 84.050.000 (ms)                 |
| time.tz     | Long  | 23:20:50+0100            | 1.250.000 (ms)                  |

The UPnP specification points to [2] *XML Schema*. In this standard, [3] *ISO 8601 Date And Time formats* are referenced. The mapping is not completely defined which means that this OSGi UPnP specification defines a complete mapping to Java classes. The UPnP types date, dateTime and dateTime.tz are represented as a Date object. For the date type, the hours, minutes and seconds must all be zero.

The UPnP types time and time.tz are represented as a Long object that represents the number of ms since midnight. If the time wraps to the next day due to a time zone value, then the final value must be truncated modulo 86.400.000.

See also [TYPE\\_DATE](#).

## 111.12 UPnP Exception

The UPnP Exception can be thrown when a UPnPAction is invoked. This exception contains information about the different UPnP layers. The following errors are defined:

[INVALID\\_ACTION](#) - (401) No such action could be found.

[INVALID\\_ARGS](#) - (402) Invalid argument.

[INVALID\\_SEQUENCE\\_NUMBER](#) - (403) Out of synchronization.

[INVALID\\_VARIABLE](#) - (404) State variable not found.

[DEVICE\\_INTERNAL\\_ERROR](#) - (501) Internal error.

Further errors are categorized as follows:

- *Common Action Errors* - In the range of 600-69, defined by the UPnP Forum Technical Committee.
- *Action Specific Errors* - In the range of 700-799, defined by the UPnP Forum Working Committee.
- *Non-Standard Action Specific Errors* - In the range of 800-899. Defined by vendors.

## 111.13 Configuration

In order to provide a standardized way to configure a UPnP driver bundle, the Configuration Admin property `upnp.ssdp.address` is defined.

The value is a `String[]` with a list of IP addresses, optionally followed with a colon (`'\u003A'`) and a port number. For example:

239.255.255.250:1900

Those addresses define the interfaces which the UPnP driver is operating on. If no SSDP address is specified, the default assumed will be 239.255.255.250:1900. If no port is specified, port 1900 is assumed as default.

## 111.14 Networking considerations

### 111.14.1 The UPnP Multicasts

The operating system must support multicasting on the selected network device. In certain cases, a multicasting route has to be set in the operating system routing table.

These configurations are highly dependent on the underlying operating system and beyond the scope of this specification.

## 111.15 Security

The UPnP specification is based on HTTP and uses plain text SOAP (XML) messages to control devices. For this reason, it does not provide any inherent security mechanisms. However, the UPnP specification is based on the exchange of XML files and not code. This means that at least worms and viruses cannot be implemented using the UPnP protocols.

However, a bundle registering a UPnP Device service is represented on the outside network and has the ability to communicate. The same is true for getting a UPnP Device service. It is therefore recommended that `ServicePermission[UPnPDevice|UPnPEventListener, REGISTER|GET]` be used sparingly and only for bundles that are trusted.

## 111.16 `org.osgi.service.upnp`

UPnP Package Version 1.2.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.upnp; version="[1.2,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.upnp; version="[1.2,1.3)"}
```

## 111.16.1 Summary

- UPnPAction - A UPnP action.
- UPnPDevice - Represents a UPnP device.
- UPnPEventListener - UPnP Events are mapped and delivered to applications according to the OS-Gi whiteboard model.
- UPnPException - There are several defined error situations describing UPnP problems while a control point invokes actions to UPnPDevices.
- UPnPIcon - A UPnP icon representation.
- UPnPLocalStateVariable - A local UPnP state variable which allows the value of the state variable to be queried.
- UPnPService - A representation of a UPnP Service.
- UPnPStateVariable - The meta-information of a UPnP state variable as declared in the device's service state table (SST).

## 111.16.2 public abstract UPnPAction

A UPnP action. Each UPnP service contains zero or more actions. Each action may have zero or more UPnP state variables as arguments.

### 111.16.2.1 public abstract String[][] getInputArgumentNames()

- Lists all input arguments for this action.

Each action may have zero or more input arguments.

This method must continue to return the action input argument names after the UPnP action has been removed from the network.

*Returns* Array of input argument names or {@code null} if no input arguments.

*See Also* UPnPStateVariable

### 111.16.2.2 public abstract String getName()

- Returns the action name. The action name corresponds to the {@code name} field in the {@code actionList} of the service description.
- For standard actions defined by a UPnP Forum working committee, action names must not begin with {@code X\_} nor {@code A\_}.
- For non-standard actions specified by a UPnP vendor and added to a standard service, action names must begin with {@code X\_}.

This method must continue to return the action name after the UPnP action has been removed from the network.

*Returns* Name of action, must not contain a hyphen character or a hash character

### 111.16.2.3 public abstract String[][] getOutputArgumentNames()

- List all output arguments for this action.

This method must continue to return the action output argument names after the UPnP action has been removed from the network.

*Returns* Array of output argument names or {@code null} if there are no output arguments.

*See Also* UPnPStateVariable

### 111.16.2.4 public abstract String getReturnArgumentName()

- Returns the name of the designated return argument.

One of the output arguments can be flagged as a designated return argument.

This method must continue to return the action return argument name after the UPnP action has been removed from the network.

*Returns* The name of the designated return argument or {@code null} if none is marked.

#### 111.16.2.5 **public abstract UPnPStateVariable getStateVariable(String argumentName)**

*argumentName* The name of the UPnP action argument.

- Finds the state variable associated with an argument name. Helps to resolve the association of state variables with argument names in UPnP actions.

*Returns* State variable associated with the named argument or {@code null} if there is no such argument.

*Throws* `IllegalStateException`—if the UPnP action has been removed from the network.

*See Also* `UPnPStateVariable`

#### 111.16.2.6 **public abstract Dictionary<String, Object> invoke(Dictionary<String, Object> args) throws Exception**

*args* A {@code Dictionary} of arguments. Must contain the correct set and type of arguments for this action. May be {@code null} if no input arguments exist.

- Invokes the action. The input and output arguments are both passed as {@code Dictionary} objects. Each entry in the {@code Dictionary} object has a {@code String} object as key representing the argument name and the value is the argument itself. The class of an argument value must be assignable from the class of the associated UPnP state variable. The input argument {@code Dictionary} object must contain exactly those arguments listed by {@code getInputArguments} method. The output argument {@code Dictionary} object will contain exactly those arguments listed by {@code getOutputArguments} method.

*Returns* A {@code Dictionary} with the output arguments. {@code null} if the action has no output arguments.

*Throws* `UPnPException`—A UPnP error has occurred.

`IllegalStateException`—if the UPnP action has been removed from the network.

`Exception`—The execution fails for some reason.

*See Also* `UPnPStateVariable`

### 111.16.3 **public abstract UPnPDevice**

Represents a UPnP device. For each UPnP root and embedded device, an object is registered with the framework under the {@code UPnPDevice} interface.

The relationship between a root device and its embedded devices can be deduced using the {@code UPnPDevice.CHILDREN\_UDN} and {@code UPnPDevice.PARENT\_UDN} service registration properties.

The values of the UPnP property names are defined by the UPnP Forum.

All values of the UPnP properties are obtained from the device using the device's default locale.

If an application wants to query for a set of localized property values, it has to use the method {@code UPnPDevice.getDescriptions(String locale)}.

#### 111.16.3.1 **public static final String CHILDREN\_UDN = "UPnP.device.childrenUDN"**

The property key that must be set for all devices containing other embedded devices.

The value is an array of UDNs for each of the device's children ( {@code String[]}). The array contains UDNs for the immediate descendants only.

If an embedded device in turn contains embedded devices, the latter are not included in the array.

The UPnP Specification does not encourage more than two levels of nesting.

The property is not set if the device does not contain embedded devices.

The property is of type {@code String[]}. Value is "UPnP.device.childrenUDN"

**111.16.3.2      `public static final String DEVICE_CATEGORY = "UPnP"`**

Constant for the value of the service property {@code DEVICE\_CATEGORY} used for all UPnP devices. Value is "UPnP".

*See Also* org.osgi.service.device.Constants.DEVICE\_CATEGORY

**111.16.3.3      `public static final String FRIENDLY_NAME = "UPnP.device.friendlyName"`**

Mandatory property key for a short user friendly version of the device name. The property value holds a {@code String} object with the user friendly name of the device. Value is "UPnP.device.friendlyName".

**111.16.3.4      `public static final String ID = "UPnP.device.UDN"`**

Property key for the Unique Device ID property. This property is an alias to {@code UPnPDevice.UDN}. It is merely provided for reasons of symmetry with the {@code UPnPService.ID} property. The value of the property is a {@code String} object of the Device UDN. The value of the key is "UPnP.device.UDN".

**111.16.3.5      `public static final String MANUFACTURER = "UPnP.device.manufacturer"`**

Mandatory property key for the device manufacturer's property. The property value holds a String representation of the device manufacturer's name. Value is "UPnP.device.manufacturer".

**111.16.3.6      `public static final String MANUFACTURER_URL = "UPnP.device.manufacturerURL"`**

Optional property key for a URL to the device manufacturers Web site. The value of the property is a {@code String} object representing the URL. Value is "UPnP.device.manufacturerURL".

**111.16.3.7      `public static final int MATCH_GENERIC = 1`**

Constant for the UPnP device match scale, indicating a generic match for the device. Value is 1.

**111.16.3.8      `public static final int MATCH_MANUFACTURER_MODEL = 7`**

Constant for the UPnP device match scale, indicating a match with the device model. Value is 7.

**111.16.3.9      `public static final int MATCH_MANUFACTURER_MODEL_REVISION = 15`**

Constant for the UPnP device match scale, indicating a match with the device revision. Value is 15.

**111.16.3.10      `public static final int MATCH_MANUFACTURER_MODEL_REVISION_SERIAL = 31`**

Constant for the UPnP device match scale, indicating a match with the device revision and the serial number. Value is 31.

**111.16.3.11      `public static final int MATCH_TYPE = 3`**

Constant for the UPnP device match scale, indicating a match with the device type. Value is 3.

**111.16.3.12      `public static final String MODEL_DESCRIPTION = "UPnP.device.modelDescription"`**

Optional (but recommended) property key for a {@code String} object with a long description of the device for the end user. The value is "UPnP.device.modelDescription".

**111.16.3.13      `public static final String MODEL_NAME = "UPnP.device.modelName"`**

Mandatory property key for the device model name. The property value holds a {@code String} object giving more information about the device model. Value is "UPnP.device.modelName".

**111.16.3.14      public static final String MODEL\_NUMBER = "UPnP.device.modelNumber"**

Optional (but recommended) property key for a {@code String} class typed property holding the model number of the device. Value is "UPnP.device.modelNumber".

**111.16.3.15      public static final String MODEL\_URL = "UPnP.device.modelURL"**

Optional property key for a {@code String} typed property holding a string representing the URL to the Web site for this model. Value is "UPnP.device.modelURL".

**111.16.3.16      public static final String PARENT\_UDN = "UPnP.device.parentUDN"**

The property key that must be set for all embedded devices. It contains the UDN of the parent device. The property is not set for root devices. The value is "UPnP.device.parentUDN".

**111.16.3.17      public static final String PRESENTATION\_URL = "UPnP.presentationURL"**

Optional (but recommended) property key for a {@code String} typed property holding a string representing the URL to a device representation Web page. Value is "UPnP.presentationURL".

**111.16.3.18      public static final String SERIAL\_NUMBER = "UPnP.device.serialNumber"**

Optional (but recommended) property key for a {@code String} typed property holding the serial number of the device. Value is "UPnP.device.serialNumber".

**111.16.3.19      public static final String TYPE = "UPnP.device.type"**

Property key for the UPnP Device Type property. Some standard property values are defined by the Universal Plug and Play Forum. The type string also includes a version number as defined in the UPnP specification. This property must be set.

For standard devices defined by a UPnP Forum working committee, this must consist of the following components in the given order separated by colons:

- {@code urn}
- schemas-upnp-org
- {@code device}
- a device type suffix
- an integer device version

For non-standard devices specified by UPnP vendors following components must be specified in the given order separated by colons:

- {@code urn}
- an ICANN domain name owned by the vendor
- {@code device}
- a device type suffix
- an integer device version

To allow for backward compatibility the UPnP driver must automatically generate additional Device Type property entries for smaller versions than the current one. If for example a device announces its type as version 3, then properties for versions 2 and 1 must be automatically generated.

In the case of exporting a UPnPDevice, the highest available version must be announced on the network.

Syntax Example: {@code urn:schemas-upnp-org:device:deviceType:v}

The value is "UPnP.device.type".

**111.16.3.20      public static final String UDN = "UPnP.device.UDN"**

Property key for the Unique Device Name (UDN) property. It is the unique identifier of an instance of a {@code UPnPDevice}. The value of the property is a {@code String} object of the Device UDN. Value of the key is "UPnP.device.UDN". This property must be set.

**111.16.3.21      public static final String UPC = "UPnP.device.UPC"**

Optional property key for a {@code String} typed property holding the Universal Product Code (UPC) of the device. Value is "UPnP.device.UPC".

**111.16.3.22      public static final String UPNP\_EXPORT = "UPnP.export"**

The {@code UPnP.export} service property is a hint that marks a device to be picked up and exported by the UPnP Service. Imported devices do not have this property set. The registered property requires no value.

The UPNP\_EXPORT string is "UPnP.export".

**111.16.3.23      public abstract Dictionary<String, Object> getDescriptions(String locale)**

*locale* A language tag as defined by RFC 1766 and maintained by ISO 639. Examples include "{@code de}", "{@code en}" or "{@code en-US}". The default locale of the device is specified by passing a {@code null} argument.

- Get a set of localized UPnP properties. The UPnP specification allows a device to present different device properties based on the client's locale. The properties used to register the UPnPDevice service in the OSGi registry are based on the device's default locale. To obtain a localized set of the properties, an application can use this method.

Not all properties might be available in all locales. This method does **not** substitute missing properties with their default locale versions.

This method must continue to return the properties after the UPnP device has been removed from the network.

*Returns* Dictionary mapping property name Strings to property value Strings

**111.16.3.24      public abstract UPnPIcon[][] getIcons(String locale)**

*locale* A language tag as defined by RFC 1766 and maintained by ISO 639. Examples include "{@code de}", "{@code en}" or "{@code en-US}". The default locale of the device is specified by passing a {@code null} argument.

- Lists all icons for this device in a given locale. The UPnP specification allows a device to present different icons based on the client's locale.

*Returns* Array of icons or null if no icons are available.

*Throws* `IllegalStateException`—if the UPnP device has been removed from the network.

**111.16.3.25      public abstract UPnPService getService(String serviceId)**

*serviceId* The service id

- Locates a specific service by its service id.

*Returns* The requested service or null if not found.

*Throws* `IllegalStateException`—if the UPnP device has been removed from the network.

**111.16.3.26      public abstract UPnPService[][] getServices()**

- Lists all services provided by this device.

*Returns* Array of services or {@code null} if no services are available.

*Throws* `IllegalStateException`—if the UPnP device has been removed from the network.

**111.16.4 public abstract UPnPEventListener**

UPnP Events are mapped and delivered to applications according to the OSGi whiteboard model. An application that wishes to be notified of events generated by a particular UPnP Device registers a service extending this interface.

The notification call from the UPnP Service to any {@code UPnPEventListener} object must be done asynchronous with respect to the originator (in a separate thread).

Upon registration of the UPnP Event Listener service with the Framework, the service is notified for each variable which it listens for with an initial event containing the current value of the variable. Subsequent notifications only happen on changes of the value of the variable.

A UPnP Event Listener service filter the events it receives. This event set is limited using a standard framework filter expression which is specified when the listener service is registered.

The filter is specified in a property named "upnp.filter" and has as a value an object of type {@code org.osgi.framework.Filter}.

When the Filter is evaluated, the following keywords are recognized as defined as literal constants in the {@code UPnPDevice} class.

The valid subset of properties for the registration of UPnP Event Listener services are:

- {@code UPnPDevice.TYPE}-- Which type of device to listen for events.
- {@code UPnPDevice.ID}-- The ID of a specific device to listen for events.
- {@code UPnPService.TYPE}-- The type of a specific service to listen for events.
- {@code UPnPService.ID}-- The ID of a specific service to listen for events.

**111.16.4.1 public static final String UPNP\_FILTER = "upnp.filter"**

Key for a service property having a value that is an object of type {@code org.osgi.framework.Filter} and that is used to limit received events.

**111.16.4.2 public abstract void notifyUPnPEvent(String deviceId, String serviceId, Dictionary<String, Object> events)**

*deviceId* ID of the device sending the events

*serviceId* ID of the service sending the events

*events* {@code Dictionary} object containing the new values for the state variables that have changed.

- Callback method that is invoked for received events. The events are collected in a {@code Dictionary} object. Each entry has a {@code String} key representing the event name (= state variable name) and the new value of the state variable. The class of the value object must match the class specified by the UPnP State Variable associated with the event. This method must be called asynchronously

**111.16.5 public class UPnPException extends Exception**

There are several defined error situations describing UPnP problems while a control point invokes actions to UPnPDevices.

*Since* @since 1.1

**111.16.5.1 public static final int DEVICE\_INTERNAL\_ERROR = 501**

The invoked action failed during execution.

**111.16.5.2 public static final int INVALID\_ACTION = 401**

No Action found by that name at this service.

**111.16.5.3      public static final int INVALID\_ARGS = 402**

Not enough arguments, too many arguments with a specific name, or one of more of the arguments are of the wrong type.

**111.16.5.4      public static final int INVALID\_SEQUENCE\_NUMBER = 403**

The different end-points are no longer in synchronization.

**111.16.5.5      public static final int INVALID\_VARIABLE = 404**

Refers to a non existing variable.

**111.16.5.6      public UPnPException(int errorCode, String errorDescription)**

*errorCode* error code which defined by UPnP Device Architecture V1.0.

*errorDescription* error description which explain the type of problem.

- This constructor creates a {@code UPnPException} on the specified error code and error description.

**111.16.5.7      public UPnPException(int errorCode, String errorDescription, Throwable errorCause)**

*errorCode* error code which defined by UPnP Device Architecture V1.0.

*errorDescription* error description which explain the type of the problem.

*errorCause* cause of that {@code UPnPException}.

- This constructor creates a {@code UPnPException} on the specified error code, error description and error cause.

*Since* @since 1.2

**111.16.5.8      public int getUPnPError\_Code()**

- Returns the UPnPError Code occurred by UPnPDevices during invocation.

*Returns* The UPnPErrorCode defined by a UPnP Forum working committee or specified by a UPnP vendor.

*Deprecated* @deprecated As of 1.2. Replaced by {@link #getUPnPErrorCode()}.

**111.16.5.9      public int getUPnPErrorCode()**

- Returns the UPnP Error Code occurred by UPnPDevices during invocation.

*Returns* The UPnPErrorCode defined by a UPnP Forum working committee or specified by a UPnP vendor.

*Since* @since 1.2

**111.16.6      public abstract UPnPIcon**

A UPnP icon representation. Each UPnP device can contain zero or more icons.

**111.16.6.1      public abstract int getDepth()**

- Returns the color depth of the icon in bits.

This method must continue to return the icon depth after the UPnP device has been removed from the network.

*Returns* The color depth in bits. If the actual color depth of the icon is unknown, -1 is returned.

**111.16.6.2      public abstract int getHeight()**

- Returns the height of the icon in pixels. If the actual height of the icon is unknown, -1 is returned.

This method must continue to return the icon height after the UPnP device has been removed from the network.

*Returns* The height in pixels, or -1 if unknown.

**111.16.6.3      public abstract InputStream getInputStream() throws IOException**

- Returns an {@code InputStream} object for the icon data. The {@code InputStream} object provides a way for a client to read the actual icon graphics data. The number of bytes available from this {@code InputStream} object can be determined via the {@code getSize()} method. The format of the data encoded can be determined by the MIME type available via the {@code getMimeType()} method.

*Returns* An InputStream to read the icon graphics data from.

*Throws* IOException – If the {@code InputStream} cannot be returned.

IllegalStateException – if the UPnP device has been removed from the network.

*See Also* UPnPIcon.getMimeType()

**111.16.6.4      public abstract String getMimeType()**

- Returns the MIME type of the icon. This method returns the format in which the icon graphics, read from the {@code InputStream} object obtained by the {@code getInputStream()} method, is encoded.

The format of the returned string is in accordance to RFC2046. A list of valid MIME types is maintained by the IANA [<http://www.iana.org/assignments/media-types/>].

Typical values returned include: "image/jpeg" or "image/gif"

This method must continue to return the icon MIME type after the UPnP device has been removed from the network.

*Returns* The MIME type of the encoded icon.

**111.16.6.5      public abstract int getSize()**

- Returns the size of the icon in bytes. This method returns the number of bytes of the icon available to read from the {@code InputStream} object obtained by the {@code getInputStream()} method. If the actual size can not be determined, -1 is returned.

*Returns* The icon size in bytes, or -1 if the size is unknown.

*Throws* IllegalStateException – if the UPnP device has been removed from the network.

**111.16.6.6      public abstract int getWidth()**

- Returns the width of the icon in pixels. If the actual width of the icon is unknown, -1 is returned.

This method must continue to return the icon width after the UPnP device has been removed from the network.

*Returns* The width in pixels, or -1 if unknown.

**111.16.7      public abstract UPnPLocalStateVariable  
extends UPnPStateVariable**

A local UPnP state variable which allows the value of the state variable to be queried.

*Since* @since 1.1

**111.16.7.1      public abstract Object getCurrentValue()**

- This method will keep the current values of UPnPStateVariables of a UPnPDevice whenever UPnPStateVariable's value is changed, this method must be called.

*Returns* {@code Object} current value of UPnPStateVariable. If the current value is initialized with the default value defined UPnP service description.

*Throws* `IllegalStateException` – If the UPnP state variable has been removed.

### 111.16.8 **public abstract UPnPService**

A representation of a UPnP Service. Each UPnP device contains zero or more services. The UPnP description for a service defines actions, their arguments, and event characteristics.

#### 111.16.8.1 **public static final String ID = "UPnP.service.id"**

Property key for the optional service id. The service id property is used when registering UPnP Device services or UPnP Event Listener services. The value of the property contains a {@code String} array ({@code String[]}) of service ids. A UPnP Device service can thus announce what service ids it contains. A UPnP Event Listener service can announce for what UPnP service ids it wants notifications. A service id does **not** have to be universally unique. It must be unique only within a device. A {@code null} value is a wildcard, matching **all** services. The value is "UPnP.service.id".

#### 111.16.8.2 **public static final String TYPE = "UPnP.service.type"**

Property key for the optional service type uri. The service type property is used when registering UPnP Device services and UPnP Event Listener services. The property contains a {@code String} array ({@code String[]}) of service types. A UPnP Device service can thus announce what types of services it contains. A UPnP Event Listener service can announce for what type of UPnP services it wants notifications. The service version is encoded in the type string as specified in the UPnP specification. A {@code null} value is a wildcard, matching **all** service types. Value is "UPnP.service.type".

*See Also* `UPnPService.getType()`

#### 111.16.8.3 **public abstract UPnPAction getAction(String name)**

*name* Name of action. Must not contain hyphen or hash characters. Should be < 32 characters.

- Locates a specific action by name. Looks up an action by its name.

*Returns* The requested action or {@code null} if no action is found.

*Throws* `IllegalStateException` – if the UPnP service has been removed from the network.

#### 111.16.8.4 **public abstract UPnPAction[][] getActions()**

- Lists all actions provided by this service.

*Returns* Array of actions ({@code UPnPAction[]}) or {@code null} if no actions are defined for this service.

*Throws* `IllegalStateException` – if the UPnP service has been removed from the network.

#### 111.16.8.5 **public abstract String getid()**

- Returns the {@code serviceId} field in the UPnP service description.

For standard services defined by a UPnP Forum working committee, the serviceId must contain the following components in the indicated order:

- {@code urn:upnp-org:serviceId:}
- service ID suffix

Example: {@code urn:upnp-org:serviceId:serviceID}.

Note that {@code upnp-org} is used instead of {@code schemas-upnp-org} in this example because an XML schema is not defined for each serviceId.

For non-standard services specified by UPnP vendors, the serviceId must contain the following components in the indicated order:

- {@code urn:}
- ICANN domain name owned by the vendor

- {@code :serviceId:}
- service ID suffix

Example: {@code urn:domain-name:serviceId:serviceID}.

This method must continue to return the service id after the UPnP service has been removed from the network.

*Returns* The service ID suffix defined by a UPnP Forum working committee or specified by a UPnP vendor. Must be <= 64 characters. Single URI.

#### 111.16.8.6 **public abstract UPnPStateVariable getStateVariable(String name)**

*name* Name of the State Variable

- Gets a {@code UPnPStateVariable} objects provided by this service by name

*Returns* State variable or {@code null} if no such state variable exists for this service.

*Throws* `IllegalStateException`— if the UPnP service has been removed from the network.

#### 111.16.8.7 **public abstract UPnPStateVariable[][] getStateVariables()**

- Lists all {@code UPnPStateVariable} objects provided by this service.

*Returns* Array of state variables or {@code null} if none are defined for this service.

*Throws* `IllegalStateException`— if the UPnP service has been removed from the network.

#### 111.16.8.8 **public abstract String getType()**

- Returns the {@code serviceType} field in the UPnP service description.

For standard services defined by a UPnP Forum working committee, the serviceType must contain the following components in the indicated order:

- {@code urn:schemas-upnp-org:service:}
- service type suffix:
- integer service version

Example: {@code urn:schemas-upnp-org:service:serviceType:v}.

For non-standard services specified by UPnP vendors, the {@code serviceType} must contain the following components in the indicated order:

- {@code urn:}
- ICANN domain name owned by the vendor
- {@code :service:}
- service type suffix:
- integer service version

Example: {@code urn:domain-name:service:serviceType:v}.

This method must continue to return the service type after the UPnP service has been removed from the network.

*Returns* The service type suffix defined by a UPnP Forum working committee or specified by a UPnP vendor. Must be <= 64 characters, not including the version suffix and separating colon. Single URI.

#### 111.16.8.9 **public abstract String getVersion()**

- Returns the version suffix encoded in the {@code serviceType} field in the UPnP service description.

This method must continue to return the service version after the UPnP service has been removed from the network.

*Returns* The integer service version defined by a UPnP Forum working committee or specified by a UPnP vendor.

### 111.16.9 **public abstract UPnPStateVariable**

The meta-information of a UPnP state variable as declared in the device's service state table (SST).

Method calls to interact with a device (e.g. {@code UPnPAction.invoke(...)} use this class to encapsulate meta information about the input and output arguments.

The actual values of the arguments are passed as Java objects. The mapping of types from UPnP data types to Java data types is described with the field definitions.

#### 111.16.9.1 **public static final String TYPE\_BIN\_BASE64 = "bin.base64"**

MIME-style Base64 encoded binary BLOB.

Takes 3 Bytes, splits them into 4 parts, and maps each 6 bit piece to an octet. (3 octets are encoded as 4.) No limit on size.

Mapped to {@code byte[]} object. The Java byte array will hold the decoded content of the BLOB.

#### 111.16.9.2 **public static final String TYPE\_BIN\_HEX = "bin.hex"**

Hexadecimal digits representing octets.

Treats each nibble as a hex digit and encodes as a separate Byte. (1 octet is encoded as 2.) No limit on size.

Mapped to {@code byte[]} object. The Java byte array will hold the decoded content of the BLOB.

#### 111.16.9.3 **public static final String TYPE\_BOOLEAN = "boolean"**

True or false.

Mapped to {@code Boolean} object.

#### 111.16.9.4 **public static final String TYPE\_CHAR = "char"**

Unicode string.

One character long.

Mapped to {@code Character} object.

#### 111.16.9.5 **public static final String TYPE\_DATE = "date"**

A calendar date.

Date in a subset of ISO 8601 format without time data.

See <http://www.w3.org/TR/xmlschema-2/#date> [<http://www.w3.org/TR/xmlschema-2/#date>].

Mapped to {@code java.util.Date} object. Always 00:00 hours.

#### 111.16.9.6 **public static final String TYPE\_DATETIME = "dateTime"**

A specific instant of time.

Date in ISO 8601 format with optional time but no time zone.

See <http://www.w3.org/TR/xmlschema-2/#dateTime> [<http://www.w3.org/TR/xmlschema-2/#dateTime>].

Mapped to {@code java.util.Date} object using default time zone.

#### 111.16.9.7 **public static final String TYPE\_DATETIME\_TZ = "dateTime.tz"**

A specific instant of time.

Date in ISO 8601 format with optional time and optional time zone.

See <http://www.w3.org/TR/xmlschema-2/#dateTime> [<http://www.w3.org/TR/xmlschema-2/#date-Time>].

Mapped to {@code java.util.Date} object adjusted to default time zone.

**111.16.9.8      `public static final String TYPE_FIXED_14_4 = "fixed.14.4"`**

Same as r8 but no more than 14 digits to the left of the decimal point and no more than 4 to the right.

Mapped to {@code Double} object.

**111.16.9.9      `public static final String TYPE_FLOAT = "float"`**

Floating-point number.

Mantissa (left of the decimal) and/or exponent may have a leading sign. Mantissa and/or exponent may have leading zeros. Decimal character in mantissa is a period, i.e., whole digits in mantissa separated from fractional digits by period. Mantissa separated from exponent by E. (No currency symbol.) (No grouping of digits in the mantissa, e.g., no commas.)

Mapped to {@code Float} object.

**111.16.9.10     `public static final String TYPE_I1 = "i1"`**

1 Byte int.

Mapped to {@code Integer} object.

**111.16.9.11     `public static final String TYPE_I2 = "i2"`**

2 Byte int.

Mapped to {@code Integer} object.

**111.16.9.12     `public static final String TYPE_I4 = "i4"`**

4 Byte int.

Must be between -2147483648 and 2147483647

Mapped to {@code Integer} object.

**111.16.9.13     `public static final String TYPE_INT = "int"`**

Integer number.

Mapped to {@code Integer} object.

**111.16.9.14     `public static final String TYPE_NUMBER = "number"`**

Same as r8.

Mapped to {@code Double} object.

**111.16.9.15     `public static final String TYPE_R4 = "r4"`**

4 Byte float.

Same format as float. Must be between 3.40282347E+38 to 1.17549435E-38.

Mapped to {@code Float} object.

**111.16.9.16     `public static final String TYPE_R8 = "r8"`**

8 Byte float.

Same format as float. Must be between -1.79769313486232E308 and -4.94065645841247E-324 for negative values, and between 4.94065645841247E-324 and 1.79769313486232E308 for positive values, i.e., IEEE 64-bit (8-Byte) double.

Mapped to {@code Double} object.

**111.16.9.17      public static final String TYPE\_STRING = "string"**

Unicode string.

No limit on length.

Mapped to {@code String} object.

**111.16.9.18      public static final String TYPE\_TIME = "time"**

An instant of time that recurs every day.

Time in a subset of ISO 8601 format with no date and no time zone.

See <http://www.w3.org/TR/xmlschema-2/#time> [<http://www.w3.org/TR/xmlschema-2/#dateTime>].

Mapped to {@code Long}. Converted to milliseconds since midnight.

**111.16.9.19      public static final String TYPE\_TIME\_TZ = "time.tz"**

An instant of time that recurs every day.

Time in a subset of ISO 8601 format with optional time zone but no date.

See <http://www.w3.org/TR/xmlschema-2/#time> [<http://www.w3.org/TR/xmlschema-2/#dateTime>].

Mapped to {@code Long} object. Converted to milliseconds since midnight and adjusted to default time zone, wrapping at 0 and 24\*60\*60\*1000.

**111.16.9.20      public static final String TYPE\_UI1 = "ui1"**

Unsigned 1 {@code Byte} int.

Mapped to an {@code Integer} object.

**111.16.9.21      public static final String TYPE\_UI2 = "ui2"**

Unsigned 2 Byte int.

Mapped to {@code Integer} object.

**111.16.9.22      public static final String TYPE\_UI4 = "ui4"**

Unsigned 4 Byte int.

Mapped to {@code Long} object.

**111.16.9.23      public static final String TYPE\_URI = "uri"**

Universal Resource Identifier.

Mapped to {@code String} object.

**111.16.9.24      public static final String TYPE\_UUID = "uuid"**

Universally Unique ID.

Hexadecimal digits representing octets. Optional embedded hyphens are ignored.

Mapped to {@code String} object.

**111.16.9.25      public abstract String[][] getAllowedValues()**

- Returns the allowed values, if defined. Allowed values can be defined only for String types.

This method must continue to return the state variable allowed values after the UPnP state variable has been removed from the network.

*Returns* The allowed values or {@code null} if not defined. Should be less than 32 characters.

#### 111.16.9.26 **public abstract Object getDefaultValue()**

- Returns the default value, if defined.

This method must continue to return the state variable default value after the UPnP state variable has been removed from the network.

*Returns* The default value or {@code null} if not defined. The type of the returned object can be determined by {@code getJavaDataType}.

#### 111.16.9.27 **public abstract Class<?> getJavaDataType()**

- Returns the Java class associated with the UPnP data type of this state variable.

Mapping between the UPnP data types and Java classes is performed according to the schema mentioned above.

|           |                             |
|-----------|-----------------------------|
| Integer   | ui1, ui2, i1, i2, i4, int   |
| Long      | ui4, time, time.tz          |
| Float     | r4, float                   |
| Double    | r8, number, fixed.14.4      |
| Character | char                        |
| String    | string, uri, uuid           |
| Date      | date, dateTime, dateTime.tz |
| Boolean   | boolean                     |
| byte[]    | bin.base64, bin.hex         |

This method must continue to return the state variable java type after the UPnP state variable has been removed from the network.

*Returns* A class object corresponding to the Java type of this argument.

#### 111.16.9.28 **public abstract Number getMaximum()**

- Returns the maximum value, if defined. Maximum values can only be defined for numeric types.

This method must continue to return the state variable maximum value after the UPnP state variable has been removed from the network.

*Returns* The maximum value or {@code null} if not defined.

#### 111.16.9.29 **public abstract Number getMinimum()**

- Returns the minimum value, if defined. Minimum values can only be defined for numeric types.

This method must continue to return the state variable minimum value after the UPnP state variable has been removed from the network.

*Returns* The minimum value or {@code null} if not defined.

#### 111.16.9.30 **public abstract String getName()**

- Returns the variable name.

- All standard variables defined by a UPnP Forum working committee must not begin with {@code X\_} nor {@code A\_}.
- All non-standard variables specified by a UPnP vendor and added to a standard service must begin with {@code X\_}.

This method must continue to return the state variable name after the UPnP state variable has been removed from the network.

*Returns* Name of state variable. Must not contain a hyphen character nor a hash character. Should be < 32 characters.

#### 111.16.9.31 **public abstract Number getStep()**

- Returns the size of an increment operation, if defined. Step sizes can be defined only for numeric types.

This method must continue to return the step size after the UPnP state variable has been removed from the network.

*Returns* The increment size or null if not defined.

#### 111.16.9.32 **public abstract String getUPnPDataType()**

- Returns the UPnP type of this state variable. Valid types are defined as constants.

This method must continue to return the state variable UPnP data type after the UPnP state variable has been removed from the network.

*Returns* The UPnP data type of this state variable, as defined in above constants.

#### 111.16.9.33 **public abstract boolean sendsEvents()**

- Tells if this StateVariable can be used as an event source. If the StateVariable is eventable, an event listener service can be registered to be notified when changes to the variable appear.

This method must continue to return the correct value after the UPnP state variable has been removed from the network.

*Returns* {@code true} if the {@code StateVariable} generates events, {@code false} otherwise.

## 111.17 References

- [1] *Open Connectivity Foundation*  
<https://openconnectivity.org/developer/specifications/upnp-resources/upnp/>
- [2] *XML Schema*  
<https://www.w3.org/TR/xmlschema-2>
- [3] *ISO 8601 Date And Time formats*  
<https://www.iso.org/iso-8601-date-and-time-format.html>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

**End Of Document**

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT